Question 1: Define Data Structure and explain its importance in computing (5 Marks)

Definition

A data structure is a way of organizing and storing data in a computer so that it can be used efficiently. It decides how data is arranged and what operations, such as insertion, deletion, searching and sorting, can be done on it.

Examples: array, linked list, stack, queue, tree, graph and hash table.

Importance in Computing

1. **Efficient storage and retrieval:** Data can be stored and found quickly. For example, binary search on a sorted array takes O(log n) time, whereas linear search takes O(n).
2. **Faster programs:** Choosing the right data structure makes an algorithm run faster and use less memory.
3. **Easy handling of large data:** Structures like trees and hash tables manage large amounts of data in an organized way.
4. **Reusability:** A data structure such as a stack or queue can be used again in many programs.
5. **Wide applications:** They are used in operating systems, databases, compilers, networking and web browsers.

Question 2: Develop a recursive solution for binary search (10 Marks)

Aim

To search for an element in a sorted array using recursive binary search.

Concept

Binary search compares the key with the middle element of a sorted array.

- If the key is equal to the middle element, the search is successful.
- If the key is smaller, the function calls itself on the left half.
- If the key is larger, the function calls itself on the right half.

The recursion stops when the element is found or when low becomes greater than high (element not present).

Algorithm

```
BinarySearch(arr, low, high, key)
1. If low > high, return -1
2. mid = (low + high) / 2
3. If arr[mid] == key, return mid
4. If key < arr[mid], return BinarySearch(arr, low, mid - 1, key)
5. Else return BinarySearch(arr, mid + 1, high, key)
```

Program (C)

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int key)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == key)
        return mid;
    else if (key < arr[mid])
        return binarySearch(arr, low, mid - 1, key);
    else
        return binarySearch(arr, mid + 1, high, key);
}

int main()
{
    int arr[] = {10, 20, 30, 40, 50, 60, 70};
    int n = 7, key = 50;

    int result = binarySearch(arr, 0, n - 1, key);

    if (result != -1)
        printf("Element found at index %d\n", result);
    else
```

```
        printf("Element not found\n");

    return 0;
}
```

Output

```
Element found at index 4
```

Complexity

- **Time:** O(log n), because the array size is halved in every call.
- **Space:** O(log n) for the recursive call stack.

Conclusion

Recursive binary search is a fast method to search a sorted array. It is much more efficient than linear search for large data.

Question 3: Explain ADT usage in database indexing (5 Marks)

Abstract Data Type (ADT)

An ADT defines a data type by the operations it supports, without showing how they are implemented. Examples are List, Stack, Queue and Map (Dictionary).

ADT in Database Indexing

A database index helps to find records quickly without scanning the entire table. It works like a Dictionary (Map) ADT, which stores a key along with the location of its record. The main operations are:

- **insert(key):** add a new entry
- **search(key):** find the record
- **delete(key):** remove an entry

The ADT can be implemented using different data structures:

Index Type	Data Structure Used	Best For
B+ Tree index	Balanced tree	Range and sorted searches
Hash index	Hash table	Exact-match searches

Advantages

6. The database uses only the operations (search, insert, delete) and does not depend on the internal structure.
7. The implementation can be changed, for example from hash table to B+ tree, without changing the rest of the system.
8. Searching becomes faster, around O(log n) instead of O(n) for a full table scan.

Conclusion

ADTs give database indexing a simple and flexible interface, while the data structure behind it provides fast searching.